

MEMORIA TRABAJO GRUPAL

Análisis Estratégico de Rendimiento y Optimización de Talento en La Liga

Autores: Jorge Hidalgo y Antonio Barrio.

Fecha: 15 de Mayo de 2026.

Grado: Ingeniería de las Tecnologías de la Información.

1. INTRODUCCIÓN Y FILOSOFÍA DEL PROYECTO

1.1. Contexto: Acabar con la intuición deportiva

El fútbol profesional ha dejado de ser un deporte basado únicamente en la intuición para convertirse en una industria que usa la inteligencia científica para llevar a cabo sus labores. El proyecto **AEROT-LL** nace de la intención de aplicar la filosofía **Moneyball**, creada por Billy Beane, al contexto específico de La Liga española, con el fin de ayudar a los clubes españoles a mejorar sus análisis y, por ende, a mejorar sus economías y proyectos deportivos.

Históricamente, los clubes han basado sus fichajes en emociones o en el rendimiento inmediato del jugador en un puñado de partidos, realizando estos “a ojo” sin ningún tipo de rigor científico ni estadístico. Con **AEROT-LL**, hemos diseñado una herramienta que permite a una dirección deportiva tomar decisiones basadas en datos fríos, buscando el **valor oculto**; es decir, jugadores que el mercado actual infravalora pero que nuestras métricas de Inteligencia Artificial detectan por la utilidad que realmente tiene.

1.2. Objetivos Estratégicos

Para alcanzar esta meta, nos propusimos tres objetivos estratégicos fundamentales:

1. **Eliminar el sesgo del mercado:** Crear una métrica propia que mida la valoración real del jugador denominada **Calidad IA**.
2. **Maximizar el retorno de inversión:** Tratar de detectar jóvenes talentos y jugadores en crecimiento antes de que su caché o precio explote, utilizaremos para ello una métrica propia que mida el potencial que vaya a tener dicho jugador denominada **Potencial IA**.
3. **Garantizar el encaje táctico en el equipo:** Rechazar rotundamente la idea de fichar por “nombre” o “fama” y preferiblemente adoptar la idea de adquirir “perfiles” que cubran, ya sea, posibles carencias como la idea de juego del equipo que use la herramienta.

2. ARQUITECTURA DE DATOS Y PROCESO ETL

2.1. Ingesta de datos iniciales desde SQLite

El proyecto se apoya en la European Soccer Database de Kaggle. Inicialmente, la información estaba dispersa en tablas relacionales dentro de un archivo SQLite, así que utilizamos la librería `sqlite3` para realizar consultas SQL que logren unificar atributos de jugadores, resultados de partidos e incluso tácticas de los equipos.

Además, para garantizar la calidad de los datos, eliminamos registros de jugadores duplicados de años anteriores. Conservamos únicamente las estadísticas más recientes de cada jugador mediante la opción `keep` y el parámetro `first` (`keep=first`) y descartamos cualquier fila que contenga valores nulos con el método `dropna()`.

```
import sqlite3
import pandas as pd
conn = sqlite3.connect('database.sqlite')
df_names = pd.read_sql("SELECT team_api_id, team_long_name FROM Team", conn)
query_p = """
SELECT p.player_name, p.birthday, pa.* FROM Player_Attributes pa
JOIN Player p ON pa.player_api_id = p.player_api_id
ORDER BY pa.date DESC
"""
df_players = pd.read_sql_query(query_p, conn)
df_players = df_players.drop_duplicates(subset=['player_api_id'], keep='first').dropna()
```

2.2. El problema de la trazabilidad

Uno de los principales problemas que nos encontramos y resolvimos fue determinar el equipo actual de cada jugador. En el dataset original, los partidos se guardan con 22 columnas para los jugadores (desde `home_player_1` hasta `away_player_11`). Esta estructura nos impedía saber de forma directa dónde jugaba un jugador actualmente.

Así que, para solucionarlo, aplicamos un proceso de despivotamiento mediante la función `pd.melt`.

Transformamos las 22 columnas en una única columna, lo que nos permitió:

- Asociar cada ID de jugador con un ID de equipo y una fecha.
- Filtrar por la fecha más reciente para obtener el **club actual** del jugador mediante la opción `keep` aplicándole el parámetro `last` (`keep = last`).
- Gestionar a aquellos jugadores que aparecían sin equipo mediante el método `fillna()`, etiquetándolos como "Retirado / Agente Libre Real", limpiando así los valores nulos.

```
h_melt = df_global_matches.melt(id_vars=['date', 'home_team_api_id'], value_vars=[f'home_player_{i}'
                                                                                for i in range(1, 12)], value_name='player_api_id').dropna()
h_melt = h_melt.rename(columns={'home_team_api_id': 'team_api_id'})
a_melt = df_global_matches.melt(id_vars=['date', 'away_team_api_id'], value_vars=[f'away_player_{i}'
                                                                                for i in range(1, 12)], value_name='player_api_id').dropna()
a_melt = a_melt.rename(columns={'away_team_api_id': 'team_api_id'})
```

```

all_players_teams = pd.concat([h_melt, a_melt])
all_players_teams['date'] = pd.to_datetime(all_players_teams['date'])
latest_team = all_players_teams.sort_values('date').drop_duplicates(subset=['player_api_id'], keep='last')
latest_team = pd.merge(latest_team, df_names, on='team_api_id', how='left')
latest_team = latest_team[['player_api_id',
                           'team_long_name']].rename(columns={'team_long_name': 'Equipo Actual'})

df_f = pd.merge(df_f, latest_team, on='player_api_id', how='left')
df_f['Equipo Actual'] = df_f['Equipo Actual'].fillna('Retirado / Agente Libre Real')

```

3. MACHINE LEARNING

3.1. Justificación del Modelo Random Forest

Inicialmente, consideramos usar una regresión lineal simple para valorar a los jugadores. Sin embargo, pronto nos dimos cuenta de que el rendimiento de un futbolista no se trata de una línea recta; es una combinación compleja de variables que interactúan entre sí. Por ello, tuvimos que optar por entrenar un modelo de **Random Forest Regressor** con scikit-learn.

Este modelo es mucho más preciso porque analiza cómo atributos como la visión de juego, el control de balón o las reacciones ágiles impactan en la media oficial que le da el mercado al jugador. Al usar 100 estimadores en nuestro modelo (`n_estimators=100`), conseguimos una valoración técnica (Calidad IA) bastante robusta, que nos sirve para detectar cuándo un jugador tiene una media oficial que no hace justicia a sus habilidades auténticas.

```

features = ['reactions', 'ball_control', 'short_passing', 'long_passing',
            'shot_power', 'vision', 'dribbling', 'long_shots', 'curve']
model = RandomForestRegressor(n_estimators=100, random_state=42,
                              n_jobs=-1).fit(df_outfield[features], df_outfield['overall_rating'])
df_outfield['valoracion_tecnica'] = model.predict(df_outfield[features])

```

3.2. Segmentación lógica por posición

Antes de proceder al entrenamiento del modelo mostrado anteriormente, detectamos la necesidad de realizar una segmentación lógica entre porteros y jugadores de campo. Dado que las aptitudes de un guardameta son incomparables con las de un jugador de campo, incluirlos en el mismo modelo de Machine Learning pondría en riesgo la precisión de las predicciones.

Por ello, implementamos una lógica de filtrado técnico en Python:

- **Guardametas:** Filtramos a los porteros utilizando el atributo “gk_diving” con un umbral superior a 40. Para ellos, diseñamos una valoración técnica específica basada en la media aritmética de sus cinco atributos clave.

```

df_gk = df_players[df_players['gk_diving'] > 40].copy()
df_gk['valoracion_tecnica'] = df_gk[['gk_diving', 'gk_handling',
                                     'gk_kicking', 'gk_positioning', 'gk_reflexes']].mean(axis=1)
df_gk['rol_principal'] = 'Portero'

```

- **Jugadores de Campo:** El resto de los futbolistas se agruparon en un dataframe independiente para ser procesados por el algoritmo de Random Forest, asegurando que la métrica de 'Calidad IA' se calcule exclusivamente sobre atributos técnicos de juego de campo.

```
df_outfield = df_players[df_players['gk_diving'] <= 40].copy()
```

3.3. Segmentación de diferentes Roles Tácticos

Para que nuestro modelo sea útil, no podíamos analizar a todos los jugadores con la misma vara de medir. Teniendo en cuenta que existen jugadores que juegan en posiciones distintas, nos damos cuenta de que existen unos atributos que son útiles para unos jugadores y no tan útiles para otros que juegan en otras posiciones del campo.

Diseñamos entonces una lógica de asignación de roles:

- **Creador:** Ponderamos atributos como la visión de juego y el pase corto.
- **Finalizador:** Priorizamos atributos como la potencia de tiro y la finalización de tiros a portería.
- **Defensivo:** En este rol nos enfocamos en el marcaje defensivo y en las entradas del jugador en tareas defensivas.

Mediante una función lambda y el método apply, el sistema evalúa cada perfil para cada jugador y le asigna el rol que le corresponda, que será aquel donde su puntuación sea máxima, garantizando que cada jugador recomendado tenga sentido posicionalmente.

```
df_outfield['rol_principal'] = df_outfield.apply(lambda r: max({
    'Creador': r['short_passing']*0.5 + r['vision']*0.5,
    'Finalizador': r['finishing']*0.6 + r['shot_power']*0.4,
    'Defensivo': r['marking']*0.5 + r['standing_tackle']*0.5
}), key=lambda k: {
    'Creador': r['short_passing']*0.5 + r['vision']*0.5,
    'Finalizador': r['finishing']*0.6 + r['shot_power']*0.4,
    'Defensivo': r['marking']*0.5 + r['standing_tackle']*0.5
}[k]), axis=1)
```

4. POTENCIAL Y PRIORIDAD

4.1. La métrica de Potencial

No nos conformamos en el proyecto solamente con el presente de un jugador, aspiramos también a predecir su futuro. Hemos implementado una fórmula de proyección que predice el potencial a futuro del jugador, incluimos una lógica basada en la edad, nuestro código establece que los 28 años son el pico de madurez de un jugador durante toda su carrera; a partir de ahí, el potencial no puede ir a más y se iguala con su calidad actual.

Sin embargo, incluimos también la ecuación una “**Cláusula**” para los jóvenes talentos: si un jugador tiene menos de 19 años y su calidad ya es destacable (>60 sobre 100), aplicamos un factor de crecimiento de **2.0**. Esto permite que el sistema detecte a posibles futuras estrellas antes que la competencia.

```
df_f['potencial_ia'] = df_f.apply(lambda r: r['valoracion_tecnica'] + ((28 - r['edad']) *  
    (2.0 if r['edad'] < 19 and r['valoracion_tecnica'] > 60 else 1.2))  
    if r['edad'] < 28 else r['valoracion_tecnica'], axis=1)
```

4.2. La métrica de prioridad de fichaje

Para facilitar la decisión final del director deportivo del equipo que usa nuestra herramienta, hemos creado una métrica final: el índice de prioridad. Esta métrica combina:

- **Ventaja IA:** Con un peso del 60% en la métrica final, esta trata de cuánto mejor es el jugador, mediante la diferencia entre nuestra métrica de valoración técnica y lo que dice el mercado en su media oficial.
- **Crecimiento Proyectado:** Con un peso del 40% en la métrica final, esta trata de la proyección que tiene el jugador con respecto a su potencial, mediante la diferencia entre nuestra métrica de potencial y de valoración técnica.

Normalizamos este resultado mediante una escala Min-Max para que en Power BI se visualice de mejor manera, como un porcentaje de 0 a 100.

```
df_f['ventaja_ia'] = df_f['valoracion_tecnica'] - df_f['overall_rating']  
df_f['crecimiento_proyectado'] = df_f['potencial_ia'] - df_f['valoracion_tecnica']  
raw_p = (df_f['ventaja_ia'] * 0.6) + (df_f['crecimiento_proyectado'] * 0.4)  
df_f['prioridad_fichaje'] = (((raw_p - raw_p.min()) / (raw_p.max() - raw_p.min())) * 100).round(1)
```

5. ANÁLISIS DE EQUIPOS

5.1. El Concepto de Prestigio

Un error habitual de los modelos de IA en el fútbol es su falta de realismo en situaciones como recomendar un jugador de talla mundial a un equipo recién ascendido. Para evitar esta situación, hemos calculado el prestigio histórico de cada club de La Liga basándonos en su media de puntos que ha conseguido en cada temporada registrada.

```
df_puntos_nombres = pd.merge(df_puntos, df_names, left_on='team_id', right_on='team_api_id')  
df_prestigio = df_puntos_nombres.groupby('team_long_name')['puntos'].mean().reset_index().rename(  
    columns={'puntos': 'puntos_medios'})  
df_prestigio = df_prestigio.sort_values('puntos_medios', ascending=False)
```

Este índice actúa como “filtro de realidad” en nuestro código, limitando la calidad máxima del jugador que se puede recomendar según el nivel de prestigio del club.

```
puntos_club = df_prestigio[df_prestigio['team_long_name'] == equipo]['puntos_medios'].values[0]  
if puntos_club >= 75: tope_ia = 99  
elif puntos_club >= 60: tope_ia = 82  
elif puntos_club >= 45: tope_ia = 78  
else: tope_ia = 74
```

5.2. Alarmas de Rendimiento deportivo

El sistema, mediante las estadísticas deportivas y tácticas de todos los equipos y junto con un umbral crítico de **1.35 goles por partido** que establecimos, analizará si un equipo sufre algún tipo de carencia deportiva o si se mantiene como un equipo equilibrado. Cabe recalcar que 1.35 goles por partido suele ser la media de goles estándar para definir si tienes un problema o no.

Las dos posibles alertas se deben a un problema ofensivo o defensivo:

- **Coladero en Defensa:** Si los goles en contra superan el umbral, el sistema recomendará perfiles “Defensivos”
- **Falta de Gol:** Si los goles a favor no llegan a ese umbral, el sistema se centrará en recomendar perfiles “Finalizadores”

```
gravedad_defensa = gc - 1.35
gravedad_ataque = 1.35 - gf
```

```
if gravedad_defensa > gravedad_ataque and gravedad_defensa > 0:
    urgencia = f"Coladero en Defensa (Recibe {gc:.2f} p/p)"
    rol_necesario = "Defensivo"
elif gravedad_ataque > gravedad_defensa and gravedad_ataque > 0:
    urgencia = f"Falta de Gol (Marca {gf:.2f} p/p)"
    rol_necesario = "Finalizador"
else:
    urgencia = "Rendimiento Equilibrado"
    rol_necesario = "Creador"
```

6. RECOMENDADOR DINÁMICO Y AFINIDAD TÁCTICA

6.1. Simulación de Mercado Real

En el bloque de exportación, ordenamos los equipos por prestigio. Los clubes más grandes tienen la prioridad de recibir las mejores recomendaciones primero por parte del sistema.

Para dar realismo, utilizamos un conjunto llamado `jugadores_fichados`. Una vez que un jugador es recomendado a un equipo, se retira del mercado para los demás, evitando que el informe sugiera el mismo fichaje a cinco clubes distintos, dando así mucha más variedad de jugadores recomendados al sistema.

6.2. La recomendación Táctica

La recomendación final no solo depende de la calidad del jugador, sino de la **Afinidad Táctica** del jugador con el equipo (**con un 70% del peso**).

El algoritmo cruza las tácticas del equipo con los atributos y especialidades del jugador:

- **Finalizador como rol necesario:** Dependiendo de la táctica del equipo se podría recomendar un rasgo de jugador u otro, si el equipo juega a centros usualmente te

recomienda un delantero Tanque que predomine en los atributos relacionados con el estilo de centros y sino, un finalizador rápido con atributos relacionados con la velocidad y la finalización.

- **Defensor como rol necesario:** Igualmente, si un equipo juega a presión alta o con bloque bajo buscará un rasgo u otro.

```
if rol_necesario == 'Finalizador':
    if tactica.get('Tipo Centros', '') == 'Mucho':
        encaje = (mercado['jumping'] * 0.5) + (mercado['strength'] * 0.5)
        perfil = "Delantero Tanque"
    else:
        encaje = (mercado['sprint_speed'] * 0.5) + (mercado['finishing'] * 0.5)
        perfil = "Finalizador Rápido"
elif rol_necesario == 'Defensivo':
    if tactica.get('Tipo Presión', '') == 'Alta':
        encaje = (mercado['sprint_speed'] * 0.5) + (mercado['aggression'] * 0.5)
        perfil = "Central Rápido"
    else:
        encaje = (mercado['marking'] * 0.5) + (mercado['strength'] * 0.5)
        perfil = "Central Físico"
else:
    encaje = (mercado['vision'] * 0.5) + (mercado['short_passing'] * 0.5)
    perfil = "Organizador Técnico"
```

```
mercado['Penalizacion_Edad'] = np.where(mercado['edad'] > 32, (mercado['edad'] - 32) * 3, 0)
mercado['Match_Total (%)'] = (mercado['Nota_Base'] + (encaje * 0.7)) - mercado['Penalizacion_Edad']
mercado['Match_Total (%)'] = mercado['Match_Total (%)'].clip(lower=0, upper=100.0).round(1)
```

```
top_5 = mercado.sort_values('Match_Total (%)', ascending=False).head(5)
jugadores_fichados.update(top_5['player_name'].tolist())
```

7. VISUALIZACIÓN EN POWER BI

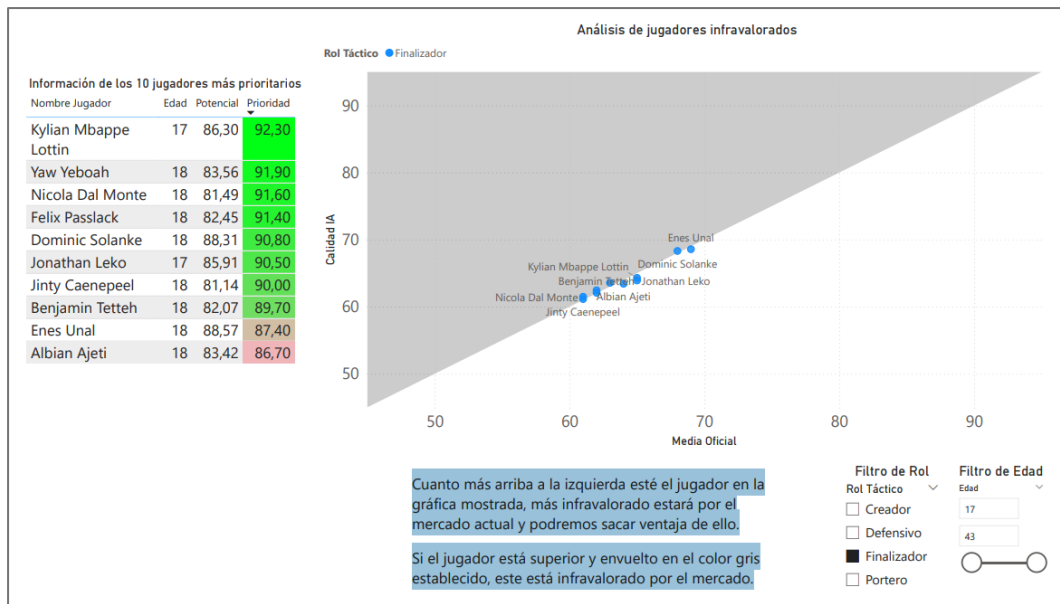
Para que AEROT-LL sea una herramienta funcional en el día a día de un club, hemos trasladado toda la inteligencia de Python a un entorno visual en **Microsoft Power BI**.

7.1. Análisis de Talento Infravalorado

Su objetivo es detectar **oportunidades de mercado** inmediatas, con esta página el director deportivo será capaz de identificar de un vistazo qué jugadores son infravalorados.

Esta página muestra un **ranking dinámico** de los 10 jugadores con mayor "Índice de Prioridad", apareciendo nombres como Kylian Mbappé o Dominic Solanke entre otros.

Además de un **gráfico de dispersión** que mide nuestra métrica de Calidad IA con la media oficial que da el mercado al jugador. Cuanto más arriba a la izquierda esté el jugador en la gráfica mostrada, más infravalorado estará por el mercado actual y podremos sacar ventaja de ello. Si el jugador está superior y envuelto en el color gris establecido, este está infravalorado por el mercado.

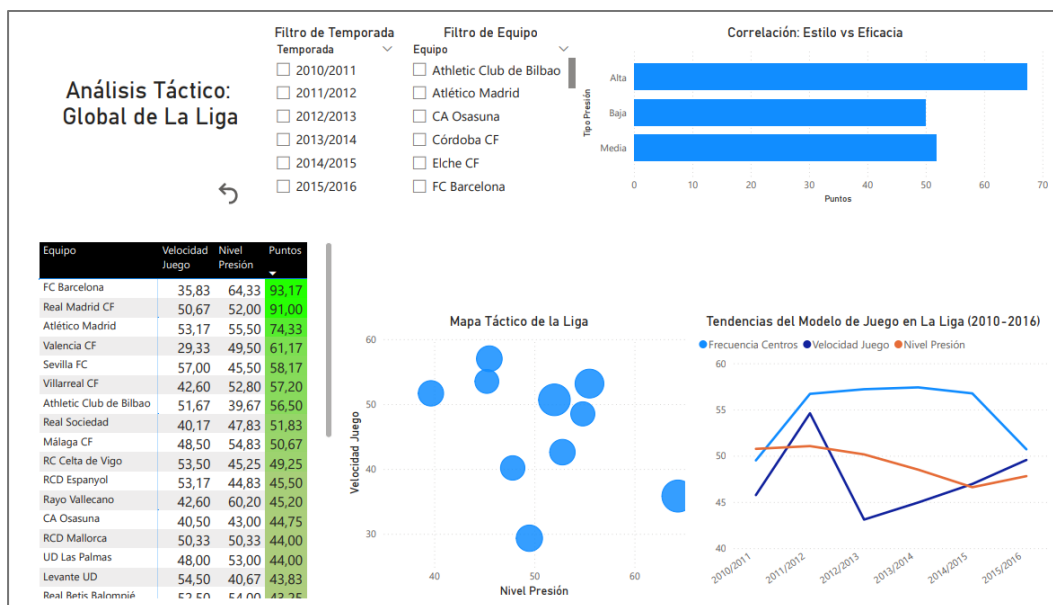


7.2. Mapa táctico y de eficacia en La Liga

Esta página sirve para entender cómo juega el propio equipo que está usando el sistema para entender el tipo de fichaje que necesita.

Muestra un **análisis histórico** (periodo 2010-2016, debido a que son los datos registrados en el dataset) de las tendencias de juego en España. Incluye una **comparativa** de "Velocidad de Juego" frente a "Nivel de Presión" por equipo, una **gráfica de correlación** entre "Estilo de Juego" y "Eficacia en Puntos" para entender el estilo de juego más eficaz para conseguir puntos y un ranking histórico de todos los equipos españoles ordenados de manera descendente por su media histórica de puntos.

Además de contar con filtros segmentados. Por equipos, para hacer un análisis individual, y por temporada.

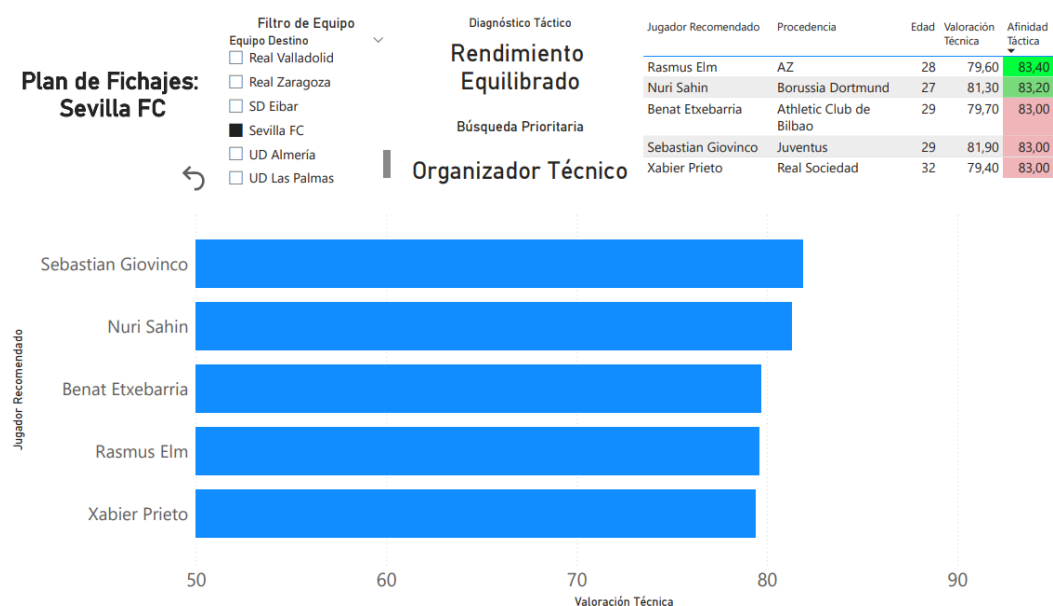


7.3. Panel de Recomendaciones

Es la página final, donde se materializa el "Recomendador" que programamos en Python.

Muestra una tabla detallada que cruza los equipos de La Liga con su "Alarma Detectada" y les asigna los 5 jugadores con mayor **Afinidad Táctica**.

Además, cuenta con filtros de segmentos para filtrar por equipo y ver los jugadores recomendados a ese equipo



8. CONCLUSIONES Y RESULTADOS

8.1. Consecución de los Objetivos

Tras finalizar el desarrollo y validación del sistema, podemos afirmar que se han cumplido íntegramente los tres objetivos fundamentales planteados al inicio del proyecto:

1. **Eliminación del sesgo del mercado:** Se ha logrado sustituir la intuición humana con métricas objetivas, demostrando que el rendimiento de un jugador puede y debe medirse mediante el impacto real de sus atributos técnicos, y no por su impacto mediático.
2. **Maximización del presupuesto:** A través de la métrica *Índice de Prioridad de Fichaje* y el entorno visual de Power BI, el sistema detecta ineficiencias en el mercado, localizando perfiles con un alto potencial de crecimiento y perfiles que no reciben la atención que se merecen, mientras se aproveche esta situación se estará maximizando el presupuesto deportivo.
3. **Garantía de encaje táctico:** No nos hemos limitado simplemente a recomendar "buenos jugadores". El algoritmo de *Afinidad Táctica Total* asegura que la inversión del club responda a una necesidad real (alarma defensiva u ofensiva) y encaje en la pizarra del equipo.

8.2. Logros Técnicos y Científicos

El proyecto ha superado con éxito retos técnicos de alta complejidad:

- **Procesamiento Masivo y Trazabilidad (ETL):** Se ha procesado una base de datos relacional masiva (>25.000 partidos históricos). Mediante técnicas avanzadas de despivotamiento, se ha resuelto el complejo problema de la trazabilidad de los jugadores para identificar su club actual.
- **Superioridad del Machine Learning:** La decisión de pivotar desde una regresión lineal inicial hacia un modelo **Random Forest Regressor** ha sido un acierto crítico en el devenir del proyecto. El algoritmo ha demostrado una capacidad superior para capturar las relaciones no lineales entre los atributos físicos/técnicos y el rendimiento global del futbolista.
- **Despliegue en Business Intelligence:** El verdadero éxito del sistema no es solo calcular predicciones o métricas complejas, sino hacerlas accesibles. Hemos encapsulado todo en un Cuadro de Mando interactivo e intuitivo, de este modo, cualquier Director Deportivo, sin necesidad de saber programar, dispone de una herramienta en donde puede monitorizar el mercado, diagnosticar las carencias de su equipo y ejecutar decisiones estratégicas de fichajes.

8.3. Líneas de Trabajo Futuro

AEROT-LL ha demostrado ser una prueba de concepto escalable y altamente funcional. Para acercar la herramienta a un producto final comercializable para equipos de élite, proponemos las siguientes vías de desarrollo:

- **Integración del *Fair Play* Financiero:** Cruzar las recomendaciones de la IA con los topes salariales oficiales de La Liga y el coste real de los traspasos, optimizando no solo el talento, sino el apartado económico del club.
- **Integración de APIs en tiempo real:** Conectarse a proveedores de datos en vivo para que los atributos de los jugadores y las alarmas de los equipos se actualicen jornada a jornada, y no solo a final de temporada.
- **Exploración Psicológica:** Incorporar modelos de análisis de texto para leer informes cualitativos que muestren datos como la actitud en el vestuario, el liderazgo de un jugador, su salud mental, etc., fusionando así el dato cuantitativo de nuestro modelo con el perfil psicológico del futbolista.